Part 0

I want to build a simple web tool to help people track their books they have read and provide them with new recommendations. What are 3 common problems people face when tracking their books, and what are 3 simple features a basic reading tracker and recommender should have

For your computer science final project, a reading tracker and book recommender is a great idea because it solves a real problem and lets you demonstrate CS1 concepts like user input, conditionals, lists, and data processing.

3 common problems readers face

1. Forgetting which books they've read. People may forget a book's title, whether they finished it, or what they thought about it.
2. Having trouble choosing their next book. With so many books available, readers may struggle to find something that matches their interests.
3. Losing track of their reading progress. Readers may forget which books they are currently reading, how far they've gotten, or which books they want to read next.

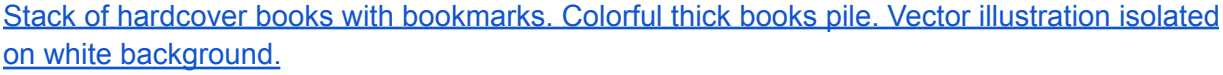
3 simple features your web tool should have

My Reading List - Notability Gallery

1. Personal book library

Let users add books by title and author, mark each book as Want to Read, Currently Reading, or Finished, and optionally give it a rating.

CS1 concepts: user input, lists, variables, and updating data.


Stack of hardcover books with bookmarks. Colorful thick books pile. Vector illustration isolated on white background.

2. Personalized book recommendations

Ask users to choose their favorite genres, such as fantasy, mystery, romance, or science fiction. Then use conditional statements to recommend books that match their preferences.

CS1 concepts: conditionals, comparisons, and filtering lists.


Open book with bookmark and check marks. Linear doodle icon. Diary with notes. Learning, knowledge, business, planning concept.

3. Reading progress dashboard

Display the number of books finished, the number currently being read, and the number on the user's reading wishlist.

CS1 concepts: counters, loops, calculations, and output.

How these features work together

Your program could follow this simple process:

1. Input

The user enters books, ratings, and favorite genres.

2. Process

The program stores book information, counts reading progress, and matches genres to recommendations.

3. Output

The user sees their book library, reading statistics, and personalized recommendations.

My recommendation: Start with these three features and keep the recommendations simple. You could create a predefined list of books with their genres and use `if` statements or loops to find matching books. That makes the project achievable while still demonstrating important programming skills.

A useful next step is to turn these features into a one-sentence project goal and a short list of requirements for your final project proposal.

For the most complex feature of my web tool how can i make it a simpler alternative

For your reading tracker and book recommender, the most complex feature is probably personalized book recommendations, because you need a way to compare a user's interests with a list of books.

A simpler alternative is to create a genre-based recommendation system.

How to simplify the feature

Original, more complex feature

Recommend books based on a user's reading history, ratings, favorite authors, and similar readers' preferences.

Simpler alternative

Let users select their favorite genre, then display books from that genre using a predefined list.

Example of how it would work

1. The user selects Fantasy from a dropdown menu.
2. The program checks a list of books and finds the ones labeled Fantasy.
3. The website displays the matching books as recommendations.

For example, your book list might contain Harry Potter, The Hobbit, and The Hunger Games, each with a genre. The program would display only books matching the user's selection.

Why this is a good choice for a CS1 project

- Input: The user selects a genre.
- Processing: A loop and conditional statements find matching books.
- Output: The website displays recommended titles.

You won't need artificial intelligence, complicated algorithms, or a large external database.

A good simplified feature requirement would be: “The program will recommend books by matching the user's selected genre to the genres stored in a predefined list of books.”

You can always add ratings or more personalized recommendations later if you have extra time.

Part 1

what are some ideas of **State Dataclass Attributes**

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

for my book reccomendation site

For your book recommendation site, you could use two dataclasses: one to represent a book and another to store the user's reading information.

1. Book dataclass

This represents an individual book that your site can recommend.

Attribute	Type	Purpose

`title`	`str`	The book's title
`author`	`str`	The book's author
`genre`	`str`	The book's genre
`rating`	`float`	The user's rating of the book

2. ReadingTracker dataclass

This represents the user's reading information and stores a collection of books.

Attribute	Type	Purpose
`username`	`str`	The user's name or username
`favorite_genre`	`str`	The genre the user enjoys most
`books_read`	`list[Book]`	A list of Book instances the user has read
`books_to_read`	`list[Book]`	A list of Book instances the user wants to read

`reading_goal`	`int`	The number of books the user wants to read

Why this works: Your `ReadingTracker` dataclass has at least four attributes, and two of them are lists of instances of your `Book` dataclass.

This structure also supports your site's main features: tracking books, setting reading goals, and recommending books based on the user's favorite genre.

You could start by deciding whether you want to include both `books_read` and `books_to_read`, or keep the project simpler with just one list of books.

what are some ideas of routes i can use for this project

For your book recommendation website, routes are the different URLs users can visit to access different pages or features.

Here are some simple route ideas for your project:

Route	Purpose
`/`	Home page with a welcome message and featured books
`/books`	Display all books in the reading tracker
`/books/<book_id>`	View details about a specific book
`/add-book`	Add a new book to the user's reading list
`/recommendations`	Display book recommendations based on the user's favorite genre

`/reading-goal`	View the user's reading goal and progress
`/favorites`	Display the user's favorite books

My recommendation for your CS1 project

Start with these four routes:

1. `/` — Home page
2. `/books` — View books the user has read
3. `/add-book` — Add a book to the tracker
4. `/recommendations` — Get book recommendations

These routes cover the main features of your website without making the project too complicated.

You could also add a `/books/<book_id>` route later if you want users to click on a book and see more information about it.

what are some ideas for pages

For your book recommendation website, you could create these pages:

Page	What it would include
Home Page	Welcome message, featured books, and links to other pages
My Reading List	A list of books the user has read, with their ratings
Add a Book	A form where users enter a book's title, author, genre, and rating
Book Recommendations	Suggested books based on the user's favorite genre

Book Details	Information about a specific book, such as its author, genre, and description
Reading Goals	The user's reading goal and how many books they have completed
Favorites	A collection of the user's favorite books

A simple version for your project

I would start with these four pages:

1. Home — Introduces the website and provides navigation.
2. My Books — Shows the books the user has read.
3. Add a Book — Allows the user to add a book to their list.
4. Recommendations — Displays books based on the user's favorite genre.

You could add a Reading Goals page later if you have extra time.

what about for user input Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes

For more information on these input types, see Drafter Documentation.

For your book recommendation website, you could use these user input components to let users add books, rate them, and choose their favorite genres.

User input components and their purposes

Component	Input example	Purpose
TextBox	Book title	Allows users to enter the title of a book they want to add to their reading list.
TextBox	Author name	Allows users to enter the author of a book.

TextArea	Book description	Allows users to enter a longer description or their personal thoughts about a book.
CheckBox	Mark as read	Allows users to indicate whether they have finished reading a book.
SelectBox	Book genre	Allows users to select a genre, such as fantasy, mystery, or romance, from a list of options.
SelectBox	Book rating	Allows users to select a rating for a book, such as 1–5 stars.

Example for your project proposal

You could describe your user input components like this:

- TextBox: Used to collect the title and author of a book when a user adds it to their reading list.
- TextArea: Used to collect a user's personal notes or review of a book.
- CheckBox: Used to indicate whether a user has finished reading a book.
- SelectBox: Used to allow users to choose a book's genre and rating from a predefined list.

These inputs would allow users to add books to their reading tracker and help your program recommend books based on their preferences.

what are some ideas of if statements i could use

For your book recommendation website, you can use `if` statements to make decisions based on the user's input. Here are some ideas that would work well for your CS1 project.

1. Book recommendations

- If the user's favorite genre is fantasy, recommend fantasy books.
- If the user's favorite genre is mystery, recommend mystery books.
- If the user has already read a book, do not recommend it again.

2. Reading tracker

- If the user marks a book as read, add it to their completed books list.
- If the user gives a book a rating of 5 stars, mark it as a favorite.

- If the user has not finished a book, keep it in their reading list.

3. Reading goals

- If the user has read at least as many books as their reading goal, display a message that they have reached their goal.
- If the user has not reached their goal, display how many more books they need to read.

4. Input validation

- If the user leaves the book title blank, display an error message.
- If the user enters a rating outside of 1–5, ask them to enter a valid rating.

Best ideas for your project

I would recommend using these three because they are simple and directly support your website's main features:

1. Genre-based recommendations: If the user selects fantasy, display fantasy books.
2. Reading progress: If the user marks a book as read, update their completed books list.
3. Reading goal: If the user reaches their goal, display a congratulatory message.

These would also give you a good opportunity to use `if`, `elif`, and `else` statements in your program.

what is an idea for a loop that can loop over dataclass instances

A great idea for your book recommendation website is to loop through a user's reading list to display all the books they have read.

For example, you could use a `for` loop to go through each `Book` dataclass instance in the `books_read` list and display its title, author, and rating.

```
None
for book in reading_tracker.books_read:    print(book.title)
print(book.author)    print(book.rating)
```

How it works:

- `reading_tracker.books_read` is a list of `Book` instances.
- The loop visits each book in the list.
- `book.title`, `book.author`, and `book.rating` access the attributes of each instance.

This would allow your website to display a user's entire reading history without needing to write separate code for every book.

Part 2

Pasted text.txtDocument

with this information and these instructions Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

At least 7 routes.

At least 5 pages.

The implementation of the State dataclass must have at least 4 fields.

At least one field must be a list of class instances or a dictionary of class instances. This means that there must be a second dataclass.

At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.

At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.

At least 3 meaningful if statements.

At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.

The site should have a legitimate purpose or functionality.

No global variables (global constantsLinks to an external site. are fine).

You must use the Drafter library.

Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above

requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus.

Once you are finished with your project code, check over the following:

Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)
Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
If a function can't be unit tested, you must note as much in a docstring or comment.
Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
Included types in your function headers
Included good docstrings Links to an external site.as the first line of your function's body
Decomposed complicated functions with helper functions
Your TA will be grading your code's quality based on the ideas above.

Repository Requirements

Your final project's GitHub repository should meet the following requirements.

Planning document should be uploaded to the repository.

Website should be deployed without error.

Site information should be set in main.py using set_site_information(...).

When changing/adding files in the repository, commit messages should be concise and descriptive, not the default "Update (filename)" message.

A video fulfilling the following requirements should be linked in the repository.

Shows functioning deployed website

Audible and visible

Accesses and explains all necessary parts of the website

All conversations with any AI should be uploaded to the repository.

The above are either present on the final rubric or are necessary for your site to be graded. help me Using the design elements developed in the prior parts of the final project (linked above), code up your website using . We recommend drafting, testing, and updating the code in Thonny. Make sure you launch your website to make sure it all works.

Once you have a working prototype of your functioning website's code, deploy it to GitHub using . Check that the deployment was successful (and that your repository is public) and troubleshoot any problems using the various methods you thought of in the prior steps.

You can still update the code after you deployed it, but you must run the deployment process again (details in the link in the prior paragraph). Make sure you make good, short, descriptive commit messages for each update after the first one (you won't be able to update those later). You don't need to fill out the *Extended description* when you commit changes, but you may. in steps

…

Absolutely! We can build your project one step at a time, using your reading tracker and book recommendation website idea.

I recommend keeping the website simple enough to understand and code for your CS1 class while making sure it meets every requirement on your rubric.

We'll use your planning document as our guide. Your original idea already has a good foundation: a `Book` dataclass, a `Library` dataclass, and features for tracking books, setting reading goals, and recommending books.

Pasted text.txt

Your website will be a reading tracker where users can:

- Add books they have read.
- View their personal book library.
- Rate books.
- Get recommendations based on their favorite genre.
- Set a reading goal and track their progress.
- View their favorite books.

You need two dataclasses.

1. Book dataclass

This represents one individual book.

Attribute	Type	Purpose
title	str	Book title
author	str	Book author
genre	str	Book genre
rating	float	User's rating

2. Library dataclass

This stores the user's reading information.

Attribute	Type	Purpose
username	str	User's name
favorite_genre	str	Favorite genre
books_read	list[Book]	Books already read
books_to_read	list[Book]	Books the user wants to read
reading_goal	int	Number of books they want to read

You can use these two dataclasses to meet the requirement for a list of class instances.

Your `State` dataclass is what Drafter uses to keep information available as users navigate your website.

I recommend these five fields:

State field	Type	How it changes
library	Library	Stores and updates the user's library
username	str	Stores the username
favorite_genre	str	Updates when the user changes their genre
reading_goal	int	Updates when the user sets a new goal
page_message	str	Changes to display success or error messages

This gives you more than the required four fields.

Important: Your `library` field contains the `Library` dataclass, which contains lists of `Book` instances. This gives you the structure needed for your loop.

You need at least seven routes. Here are eight that fit your project.

Route	Purpose
`/`	Home page
`/books`	Display all books read
`/add-book`	Add a new book
`/recommendations`	Show recommended books
`/reading-goal`	Set and track reading goals
`/favorites`	Display favorite books
`/books/<book_id>`	Display details about an individual book
`/to-read`	Display books the user wants to read

You need at least five pages. These eight routes can provide eight pages, although some routes may share a page layout.

Here is how I would organize them:

WEBSITE PAGE STRUCTURE

HOME PAGE

Welcome message and navigation links

My Books

/books

Add a Book

/add-book

Recommendations

/recommendations

Reading Goals

/reading-goal

Favorites

/favorites

Want to Read

/to-read

BOOK DETAILS

Individual book information

/books/book_id

You need at least three input fields. I recommend these five:

Input type	What the user enters
TextBox	Book title
TextBox	Author name
SelectBox	Book genre
SelectBox	Book rating (1–5)
CheckBox	Mark book as read

You could also add a TextBox for the reading goal.

You need at least three meaningful if statements.

Here are four ideas:

1. Recommendations: If the user's favorite genre is fantasy, show fantasy recommendations.
2. Favorites: If a book receives a rating of 5, add it to the favorites list.
3. Reading goals: If the number of books read reaches the reading goal, display a goal-completed message.
4. Input validation: If the user leaves the title blank, display an error message.

These all have a purpose in your website rather than just existing to satisfy the rubric.

Your loop will iterate through the `books_read` list.

For example, imagine the user has added five books.

Your loop will:

- Visit each `Book` instance.
- Access its title.
- Access its author.
- Access its rating.
- Display the information on the My Books page.

This is useful because you won't have to write separate code for every book.

Now we're ready to start implementing the website.

I recommend building it in this order:

Create your Python file

Open Thonny and create a new file named `main.py`.

Import your libraries

Import Drafter and the dataclass functionality.

Create your dataclasses

Write your `Book`, `Library`, and `State` dataclasses.

Create your initial state

Set up an initial library with some example books so you can test the website.

Create your routes

Start with the home page and then add the other routes.

Add user inputs

Build your Add a Book form and reading goal form.

Add processing

Implement your if statements and loop.

Test your website

Launch it and make sure all the routes and inputs work.

Deploy to GitHub Pages

Upload your project, commit your changes, and deploy your website.

Here is the checklist we'll use as we build.

Project progress

At least 7 routes

At least 5 pages

Book dataclass

Library dataclass

State dataclass with at least 4 fields

List of Book instances in State

At least 4 meaningfully modified State fields

At least 3 user input fields

At least 3 meaningful if statements

At least 1 meaningful loop

Legitimate website functionality

No global variables (except constants)

Uses Drafter

Functions have type annotations and docstrings

Unit tests for routes where possible

Website successfully deployed

Planning document in GitHub repository

AI conversations uploaded to repository

Video demonstration linked in repository

I suggest we work on one part at a time instead of giving you an enormous block of code that is difficult to understand or debug.

Our first task will be creating the dataclasses and initial State.

Since this is your CS1 final project, I'll explain what each part does and guide you through writing it rather than simply handing you the entire solution.

First question: Have you already learned how to create dataclasses using `@dataclass` in Python, or would you like me to walk you through that part first?

yes i have already learned that what would my first coding step be

Since you've already learned dataclasses, your first coding step should be to set up your `main.py` file and create your three dataclasses: `Book`, `Library`, and `State`.

Let's do this in small steps so you understand what you're writing.

1. Open Thonny.
2. Create a new Python file.
3. Save it as `main.py`.
4. At the top, import the libraries you'll need:

```
from dataclasses import dataclass

from drafter import *
```

Start by writing your first dataclass.

Think about the four pieces of information you want to store about each book:

- Title
- Author
- Genre
- Rating

Your goal is to create a `Book` dataclass with those four attributes.

Next, create a `Library` dataclass.

It should store:

- Username
- Favorite genre
- Books read
- Books to read
- Reading goal

Remember that `books_read` and `books_to_read` should be lists of `Book` instances.

Finally, create your `State` dataclass with these five attributes:

Attribute	Type
library	Library
username	str
favorite_genre	str

reading_goal int

page_message str

These fields will help your website remember information as users navigate between pages.

Before we move on, try writing just your `Book` dataclass in Thonny.

Your task

Create a dataclass called `Book` with four attributes:

1. `title`
2. `author`
3. `genre`
4. `rating`

Hint

Remember to use `@dataclass` above your class and include the appropriate type annotations.

Send me what you write for your `Book` dataclass, and I'll check it before we move on to the `Library` dataclass. I'll help you debug your own code rather than just writing the whole project for you!

```
from dataclasses import dataclass
from drafter import *
@dataclass
class Book:
    title: str
    author: str
```

```
    genre: str
    rating: float
@dataclass
class Library:
    username: str
    fav_genre: str
    books_read: list[Book]
    books_to_read: list[Book]
    read_goal: int
@dataclass
class State:
    library: Library
    username: str
    fav_genre: str
    read_goal: int
    page_message: str
...
```

Great job! Your three dataclasses are set up correctly, and you've included all the required attributes. You're off to a good start!

Let's review each one.

Correct Book dataclass

- `title: str` ✓
- `author: str` ✓
- `genre: str` ✓
- `rating: float` ✓

Correct Library dataclass

- `username: str` ✓

- `fav_genre: str` ✓
- `books_read: list[Book]` ✓
- `books_to_read: list[Book]` ✓
- `read_goal: int` ✓

Your lists correctly store instances of your `Book` dataclass.

Correct State dataclass

You have all five fields:

- `library`
- `username`
- `fav_genre`
- `read_goal`
- `page_message`

These satisfy the requirement of having at least four State fields.

You currently have some information stored twice:

- `username` is in both `Library` and `State`.
- `fav_genre` is in both `Library` and `State`.
- `read_goal` is in both `Library` and `State`.

This isn't necessarily wrong, but it means you will need to keep those values synchronized when they change.

For simplicity, I'd recommend eventually deciding whether those values should live in `Library` or directly in `State`.

Now we need to create your initial State.

This is important because your website needs some starting information before users can interact with it.

For example, think about:

- What should the default username be?
- What should the default favorite genre be?
- Should the user start with an empty reading list?
- What should their starting reading goal be?

Your next task: Try creating an initial `State` instance with an empty list of books and a starting reading goal of 10.

Don't worry about creating any website pages yet!

Send me your attempt, and I'll check it before we move on to your first Drafter route.

should they exist in library or state

I recommend keeping `username`, `fav_genre`, and `read_goal` inside your Library dataclass, rather than duplicating them in `State`.

Here's why:

- Library stores all the user's reading information and preferences.

- State stores the Library and any additional information needed to manage the website.

Your dataclasses could look like this conceptually:

- `title`
- `author`
- `genre`
- `rating`

- `username`
- `fav_genre`
- `books_read`
- `books_to_read`
- `read_goal`

- `library`
- `page_message`
- `current_page`
- `search_query`

This gives you four State fields without unnecessarily duplicating information.

One important thing: You'll need to meaningfully modify at least four State fields throughout your routes. So we'll make sure the additional fields have real purposes and aren't just there to satisfy the rubric.

My recommendation: Keep your `Library` dataclass exactly as you wrote it and simplify your `State` dataclass.

Next, we can work on creating your initial State instance!

what code next

Your next step is to create your initial State instance. This will give your website starting information to work with before users begin adding books.

Add this below your three dataclasses:

```
initial_state = State(
    library=Library(
        username="Reader",
        fav_genre="Fantasy",
        books_read=[],
        books_to_read=[],
        read_goal=10
    ),
    page_message="Welcome to your reading tracker!",
    current_page="Home",
    search_query=""
)
```

This assumes your updated `State` dataclass has these four fields:

```
@dataclass
class State:
    library: Library
    page_message: str
    current_page: str
    search_query: str
```

Field	Starting value	Purpose
library	Library instance	Stores reading information
page_message	Welcome message	Displays messages to the user
current_page	Home	Tracks the current page
search_query	Empty string	Stores book searches

These fields also give us opportunities to meaningfully update State throughout the website.

Once you've added this, we'll create your first Drafter route: the Home page.

It will:

- Display your website's name.
- Welcome the user.

- Show their username.
- Provide navigation to the other pages.

Try adding the initial State to your code and tell me when you're finished. Then we'll build your first page together!